

MCDI504

MACHINE LEARNING I

AVANCE DEL PROYECTO — FASE 3:

IMPLEMENTACIÓN DE MODELOS DE CLASIFICACIÓN SUPERVISADA

Nombre integrante/s: Fabian Castillo
Francisco Santelices
Renato Villazón

Curso: MCDI504 - Machine Learning I

Fecha: 18 de agosto de 2026

Grupo: GRUPO5

APARTADOS

II. Descripción del conjunto de datos y preparación para clasificación	3
2.1 Variables predictoras seleccionadas	3
2.2 Preparación analítica realizada	4
III. Análisis exploratorio inicial del conjunto de datos.....	5
3.1 Distribución de clases.....	5
3.2 Calidad de datos y valores faltantes.....	5
3.3 Patrones preliminares para clasificación.....	6
IV. Notebook F3_Clasificacion.ipynb y documentación del proceso	7
4.1 Estructura mínima sugerida del repositorio.....	7
4.2 Evidencia de código fuente sin modificación	7
V. Modelo KNN	8
5.1 Configuración mínima e implementación	8
5.2 Resultados obtenidos e interpretación inicial.....	8
VI. Modelo de árboles de decisión	9
6.1 Configuración mínima e implementación	9
6.2 Resultados obtenidos e interpretación inicial.....	9
VII. Modelo SVM.....	10
7.1 Configuración mínima e implementación	10
7.2 Resultados obtenidos e interpretación inicial.....	10
VIII. Modelo Naïve Bayes.....	11
8.1 Configuración mínima e implementación	11
8.2 Resultados obtenidos e interpretación inicial.....	11
IX. Modelo de redes neuronales	12
9.1 Configuración mínima e implementación	12
9.2 Resultados obtenidos e interpretación inicial.....	12
X. Evaluación preliminar de modelos de clasificación.....	14
10.1 Comparación analítica inicial.....	16
XI. Conclusiones.....	16
XII. Uso de fuentes, citación y referencias	18
XIII. Anexos	19
Anexo A. Evidencia resumida del código fuente	19
Anexo B. Resultados complementarios de redes neuronales.....	19
Anexo C. Checklist de alineación con rúbrica.....	Error! Bookmark not defined.

II. Descripción del conjunto de datos y preparación para clasificación

El caso aplicado utiliza el dataset Titanic cargado mediante seaborn. El problema de clasificación consiste en predecir la supervivencia de pasajeros a partir de variables demográficas, socioeconómicas y de viaje. La variable objetivo es survived, codificada como 0 para no sobrevivió y 1 para sobrevivió. Por tanto, se trata de un problema de aprendizaje supervisado de clasificación binaria.

Elemento	Descripción aplicada al caso
Dataset	Titanic, conjunto de datos clásico con información de pasajeros del Titanic.
Observaciones iniciales	891 registros observados en la carga del notebook.
Columnas iniciales	15 columnas: survived, pclass, sex, age, sibsp, parch, fare, embarked, class, who, adult_male, deck, embark_town, alive y alone.
Variable objetivo	survived: 0 = no sobrevivió; 1 = sobrevivió.
Propósito analítico	Predecir la clase de supervivencia y comparar modelos de clasificación supervisada.
Tipo de aprendizaje	Aprendizaje supervisado de clasificación, porque se cuenta con etiqueta conocida.

Flujo de trabajo para la clasificación supervisada

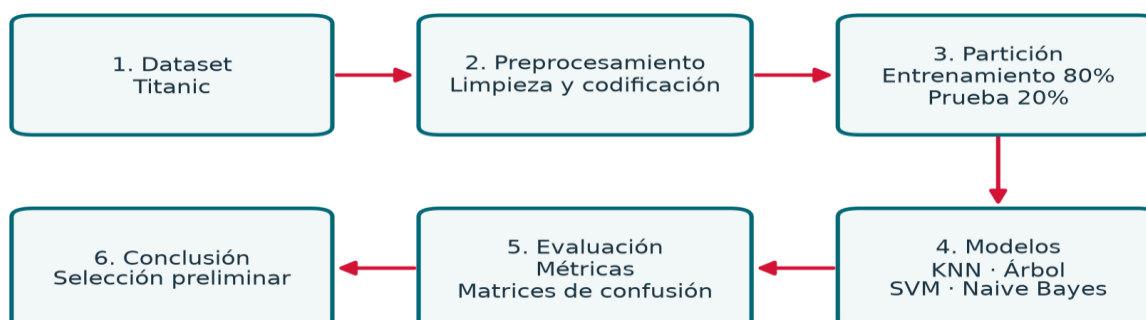


Figura 1. Flujo de trabajo aplicado en la Fase 3 del proyecto.

2.1 Variables predictoras seleccionadas

Luego de la limpieza, el código conserva variables directamente relacionadas con el problema y elimina columnas redundantes o con demasiada ausencia de datos. Después de aplicar one-hot encoding, la matriz de características queda conformada por ocho predictores.

Variable	Rol	Descripción
pclass	Predictora	Clase del pasaje: 1ª, 2ª o 3ª. Representa una aproximación al nivel socioeconómico.
age	Predictora	Edad del pasajero; los valores faltantes se imputan con la mediana.
sibsp	Predictora	Número de hermanos o cónyuges a bordo.
parch	Predictora	Número de padres o hijos a bordo.

fare	Predictora	Tarifa pagada por el pasaje.
sex_male	Predictora codificada	Variable binaria creada desde sex mediante one-hot encoding.
embarked_Q	Predictora codificada	Indicador de embarque en Queenstown.
embarked_S	Predictora codificada	Indicador de embarque en Southampton.
survived	Objetivo	Clase de supervivencia: 0 = no sobrevivió; 1 = sobrevivió.

2.2 Preparación analítica realizada

La preparación de datos sigue el flujo definido en el notebook: copia del DataFrame original, eliminación de columnas no utilizadas, imputación de valores faltantes, codificación de variables categóricas, separación entre X e y, partición entrenamiento/prueba y escalamiento para modelos sensibles a la escala. Esta preparación es coherente con el objetivo de clasificación y con la necesidad de evaluar modelos sobre datos no vistos.

Acción	Código o decisión	Justificación
Eliminación de columnas	deck, class, who, adult_male, embark_town, alive, alone	Reduce redundancia y elimina variables con demasiados valores faltantes o información derivada.
Imputación de age	Mediana	Evita perder registros; la mediana es menos sensible a valores extremos.
Imputación de embarked	Moda	Completa los 2 registros faltantes con la categoría más frecuente.
Codificación categórica	pd.get_dummies(..., drop_first=True)	Convierte sex y embarked a formato numérico binario para uso en scikit-learn.
Partición	train_test_split(test_size=0.20, random_state=42)	Reserva 20% de los datos para prueba y asegura reproducibilidad.
Escalamiento	StandardScaler para SVM y redes neuronales	Evita dominancia artificial de variables por escala y estabiliza modelos geométricos u optimizados por gradiente.

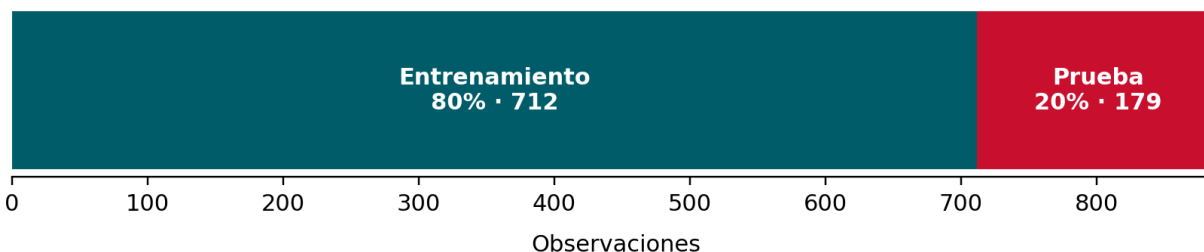


Figura 2. Partición entrenamiento/prueba utilizada en el notebook.

III. Análisis exploratorio inicial del conjunto de datos

El análisis exploratorio inicial permite comprender la estructura del problema antes de entrenar clasificadores. En este caso, la exploración se concentra en tres aspectos relevantes para la preparación de clasificación: distribución de clases, calidad de datos y comportamiento preliminar de variables predictoras.

3.1 Distribución de clases

La variable objetivo presenta dos clases: no sobrevivió y sobrevivió. En el conjunto Titanic estándar utilizado por seaborn, la clase mayoritaria corresponde a no sobrevivientes. Esta diferencia no impide el modelamiento, pero obliga a interpretar accuracy junto con métricas como precision, recall y F1-score, ya que una clase mayoritaria puede influir en la lectura global del desempeño.

Clase	Codificación	Frecuencia	Porcentaje
No sobrevivió	0	549	61,62%
Sobrevivió	1	342	38,38%
Total	—	891	100,00%

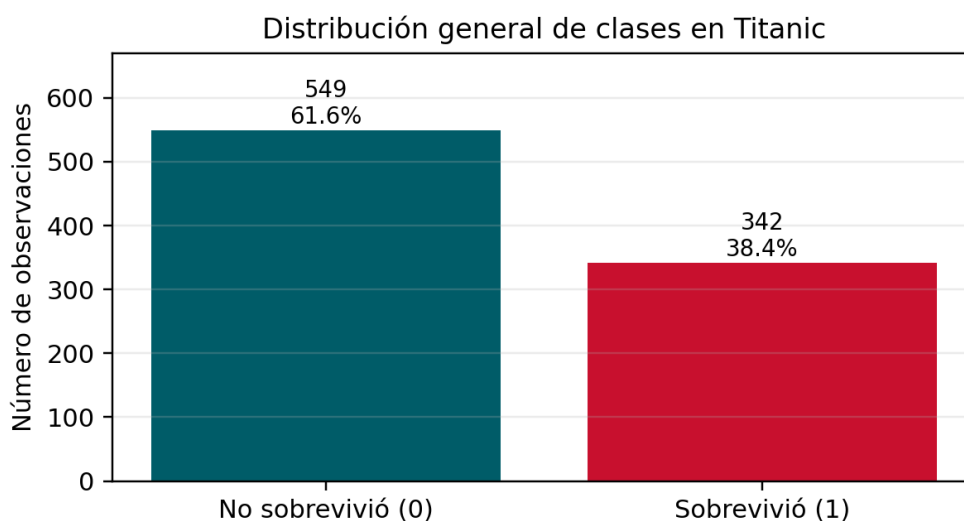


Figura 3. Distribución general de clases en el dataset Titanic.

3.2 Calidad de datos y valores faltantes

La carga inicial evidencia valores faltantes especialmente en age, embarked y deck. El código elimina deck por su alta proporción de ausencia y completa age y embarked, por lo que el DataFrame final procesado queda sin valores faltantes en sus nueve columnas. Este punto responde directamente a la exigencia de documentar calidad de datos y preparación inicial.

Variable	Valores faltantes antes	Tratamiento aplicado	Valores faltantes después
age	177	Imputación con mediana	0
embarked	2	Imputación con moda	0
deck	688	Eliminación de columna	No aplica
sex	0	One-hot encoding	0
survived	0	Se conserva como variable objetivo	0

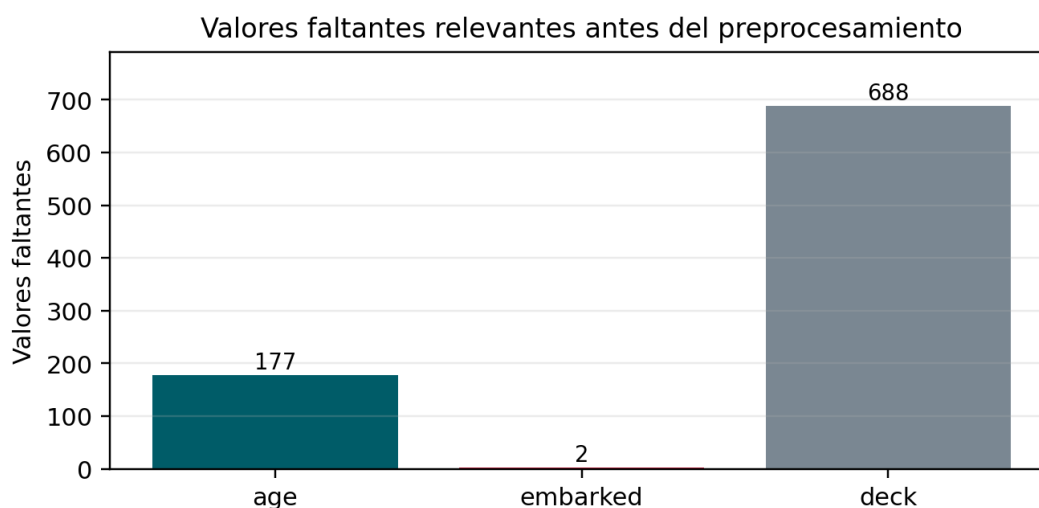


Figura 4. Valores faltantes relevantes antes del preprocesamiento.

3.3 Patrones preliminares para clasificación

La selección de variables es pertinente para un problema de supervivencia: pclass y fare aproximan condiciones socioeconómicas; sex_male captura diferencias por género; age representa una condición demográfica; sibsp y parch describen estructura familiar; embarked registra el puerto de embarque. Estos atributos entregan información potencialmente útil para separar clases, aunque su poder predictivo debe verificarse empíricamente mediante modelos y matriz de confusión.

IV. Notebook F3_Clasificacion.ipynb y documentación del proceso

El notebook constituye la evidencia técnica de la Fase 3. De acuerdo con la instrucción de la actividad, informe, notebook y repositorio deben formar un único flujo de trabajo. En este documento se respeta el código fuente entregado: no se modifican los modelos ni sus parámetros; los resultados se reportan y analizan a partir de las salidas disponibles del notebook.

Etapa del notebook	Código o acción principal	Evidencia esperada	Relación con informe
Carga de datos	<code>sns.load_dataset("titanic")</code>	Primeras filas e información estructural del DataFrame.	Secciones II y III.
Preprocesamiento	<code>drop, fillna, get_dummies</code>	Variables seleccionadas, imputación y codificación.	Secciones II y III.
Preparación X/y	<code>X = drop("survived"); y = survived</code>	Matriz de predictores y variable objetivo.	Sección II.
Partición	<code>train_test_split(test_size=0.20, random_state=42)</code>	Tamaños de entrenamiento y prueba.	Sección II.
KNN	<code>KNeighborsClassifier(n_neighbors=5, metric="minkowski", p=2)</code>	Accuracy y matriz de confusión.	Sección V.
Árbol	<code>DecisionTreeClassifier(max_depth=5, criterion="gini")</code>	Accuracy y matriz de confusión.	Sección VI.
SVM	<code>SVC(kernel="rbf", C=1)</code>	Accuracy y matriz de confusión con datos escalados.	Sección VII.
Naive Bayes	<code>GaussianNB()</code>	Accuracy y matriz de confusión.	Sección VIII.
Redes neuronales	MLPClassifier con 1, 2 y 3 capas ocultas	Accuracy, matrices de confusión e interpretación por arquitectura.	Sección IX.

4.1 Estructura mínima sugerida del repositorio

Elemento	Descripción
/notebooks/F3_Clasificacion.ipynb	Notebook con celdas narrativas, código ejecutable y resultados.
/data/README_datos.md	Descripción del dataset Titanic, variables, origen y consideraciones de uso.
/figures/	Matrices de confusión, gráficos de distribución de clases y comparación de métricas.
/reports/MCDI504_S3_ENTREGABLE_GRUPO5.pdf	Informe final exportado desde Word en formato PDF.
README.md	Instrucciones de ejecución, dependencias, versiones y resumen del flujo analítico.

Nota técnica: en la entrega final se recomienda registrar versiones de Python, pandas, seaborn, scikit-learn, NumPy y Matplotlib. Esto fortalece reproducibilidad y facilita la revisión del repositorio.

4.2 Evidencia de código fuente sin modificación

```
X = titanic_df_processed.drop('survived', axis=1)
y = titanic_df_processed['survived']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=42)
```

V. Modelo KNN

k-Nearest Neighbors es un clasificador supervisado basado en proximidad: una nueva observación se clasifica según las etiquetas de sus vecinos más cercanos en el espacio de características. El apunte de k-NN enfatiza que el parámetro k, la métrica de distancia y el escalado de variables influyen directamente en el comportamiento del modelo (Ruete, 2026a).

5.1 Configuración mínima e implementación

```
knn_model = KNeighborsClassifier(n_neighbors=5, metric='minkowski', p=2)
knn_model.fit(X_train, y_train)
y_pred_knn = knn_model.predict(X_test)
accuracy_knn = accuracy_score(y_test, y_pred_knn)
conf_matrix_knn = confusion_matrix(y_test, y_pred_knn)
```

Parámetro	Valor	Interpretación
n_neighbors	5	Considera cinco vecinos cercanos para decidir la clase.
metric	minkowski	Métrica general de distancia utilizada por scikit-learn.
p	2	Con p=2, Minkowski equivale a distancia euclidiana.
Datos usados	X_train / X_test	El código fuente entrena KNN sin aplicar StandardScaler.
Salida	survived	Clasificación binaria: 0 no sobrevivió, 1 sobrevivió.

5.2 Resultados obtenidos e interpretación inicial

Métrica / resultado	Valor
Accuracy	0,6983
Matriz de confusión	[[87, 18], [36, 38]]
Precision clase positiva	0,6786
Recall clase positiva	0,5135
F1-score clase positiva	0,5846

El KNN obtiene el desempeño más bajo entre los cuatro modelos principales. Aunque clasifica correctamente una parte importante de los no sobrevivientes, presenta 36 falsos negativos para la clase sobrevivió, lo que reduce su recall. Este comportamiento puede relacionarse con la sensibilidad del algoritmo a la escala de variables y con que el código fuente aplica KNN directamente sobre X_train sin escalamiento. Dado que no se debe modificar el código, esta observación se registra como una limitación analítica del pipeline actual.

VI. Modelo de árboles de decisión

El árbol de decisión clasifica mediante reglas jerárquicas que segmentan el espacio de variables. En clasificación, cada división busca mejorar la pureza de los nodos y cada hoja terminal asigna una clase. El recurso de Semana 3 orienta a revisar árboles como modelos basados en reglas, sobreajuste e interpretabilidad; el código implementa DecisionTreeClassifier con criterio gini y profundidad máxima controlada (Ruete, 2026g; scikit-learn Developers, 2026b).

6.1 Configuración mínima e implementación

```
dtree_model = DecisionTreeClassifier(random_state=42, max_depth=5, criterion='gini')
dtree_model.fit(X_train, y_train)
y_pred_dtree = dtree_model.predict(X_test)
accuracy_dtree = accuracy_score(y_test, y_pred_dtree)
conf_matrix_dtree = confusion_matrix(y_test, y_pred_dtree)
```

Parámetro	Valor	Interpretación
criterion	gini	Criterio para medir calidad de las divisiones.
max_depth	5	Limita la profundidad del árbol y reduce riesgo de sobreajuste.
random_state	42	Permite reproducibilidad de la estructura aprendida.
Datos usados	X_train / X_test	Modelo entrenado con características procesadas no escaladas.
Salida	survived	Clasificación binaria de supervivencia.

6.2 Resultados obtenidos e interpretación inicial

Métrica / resultado	Valor
Accuracy	0,7989
Matriz de confusión	[[95, 10], [26, 48]]
Precision clase positiva	0,8276
Recall clase positiva	0,6486
F1-score clase positiva	0,7273

El árbol mejora claramente frente a KNN. Mantiene un número bajo de falsos positivos y alcanza una precision alta para la clase sobrevivió. Sin embargo, todavía presenta 26 falsos negativos, lo que limita el recall. Su ventaja principal es la interpretabilidad mediante reglas; su limitación es que una profundidad fija puede subajustar o sobreajustar según la estructura real de los datos.

VII. Modelo SVM

Las máquinas de vectores de soporte construyen fronteras de decisión buscando maximizar el margen entre clases. El uso de kernel permite modelar separaciones no lineales; por ello, SVM es especialmente pertinente cuando la frontera entre clases no se representa adecuadamente mediante una regla lineal simple (Ruete, 2026c).

7.1 Configuración mínima e implementación

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
svm_model = SVC(kernel='rbf', C=1, random_state=42)
svm_model.fit(X_train_scaled, y_train)
y_pred_svm = svm_model.predict(X_test_scaled)
```

Parámetro	Valor	Interpretación
kernel	rbf	Permite fronteras no lineales y localmente flexibles.
C	1	Regularización estándar; equilibra margen y errores.
random_state	42	Reproducibilidad de la ejecución.
Escalamiento	StandardScaler	Necesario por sensibilidad de SVM a escalas y geometría del espacio.
Salida	survived	Clasificación binaria de supervivencia.

7.2 Resultados obtenidos e interpretación inicial

Métrica / resultado	Valor
Accuracy	0,8212
Matriz de confusión	[[95, 10], [22, 52]]
Precision clase positiva	0,8387
Recall clase positiva	0,7027
F1-score clase positiva	0,7647

SVM presenta el mejor desempeño entre los cuatro modelos exigidos por la actividad. Mantiene una precisión alta y mejora el recall respecto del árbol de decisión. La matriz de confusión muestra 52 verdaderos positivos y 22 falsos negativos para la clase sobrevivió. La combinación de kernel RBF y escalamiento permite capturar relaciones más flexibles entre variables y clase, aunque con menor interpretabilidad que un árbol.

VIII. Modelo Naïve Bayes

Naïve Bayes es un clasificador probabilístico basado en el teorema de Bayes y en el supuesto de independencia condicional entre predictores. El apunte correspondiente destaca su eficiencia computacional y su utilidad como modelo base, especialmente cuando la estructura probabilística de los datos es compatible con sus supuestos (Ruede, 2026d).

8.1 Configuración mínima e implementación

```
nb_model = GaussianNB()
nb_model.fit(X_train, y_train)
y_pred_nb = nb_model.predict(X_test)
accuracy_nb = accuracy_score(y_test, y_pred_nb)
conf_matrix_nb = confusion_matrix(y_test, y_pred_nb)
```

Parámetro / supuesto	Valor	Interpretación
Variante	GaussianNB	Supone distribución aproximadamente gaussiana de predictores continuos por clase.
priors	No especificado	El modelo estima probabilidades previas desde los datos.
var_smoothing	Valor por defecto	Estabiliza varianzas y evita problemas numéricos.
Datos usados	X_train / X_test	Modelo entrenado con características procesadas no escaladas.
Salida	survived	Clasificación binaria de supervivencia.

8.2 Resultados obtenidos e interpretación inicial

Métrica / resultado	Valor
Accuracy	0,7709
Matriz de confusión	[[84, 21], [20, 54]]
Precision clase positiva	0,7200
Recall clase positiva	0,7297
F1-score clase positiva	0,7248

Naïve Bayes presenta un accuracy inferior al árbol y a SVM, pero logra un recall de la clase sobrevivió mayor que el árbol. Esto significa que identifica más sobrevivientes, aunque también aumenta falsos positivos. Su resultado es relevante como modelo probabilístico eficiente, especialmente útil para contraste, aunque sus supuestos pueden ser restrictivos para un dataset tabular con variables correlacionadas.

IX. Modelo de redes neuronales

Las redes neuronales artificiales se incorporan como modelos supervisados de alta capacidad representacional para clasificación. Según el apunte de redes neuronales, un perceptrón multicapa organiza la información en una capa de entrada, una o más capas ocultas y una capa de salida; mediante pesos, sesgos y funciones de activación, el modelo aprende representaciones internas que pueden capturar relaciones no lineales complejas entre predictores y clases (Ruete, 2026e).

En el cuaderno de código fuente se implementan tres arquitecturas MLPClassifier sobre los datos escalados: una red con una capa oculta, una red con dos capas ocultas y una red con tres capas ocultas. En todos los casos se mantiene `activation="relu"`, `solver="adam"`, `max_iter=500` y `random_state=42`. El código muestra además una advertencia de convergencia para las redes ejecutadas, indicando que el optimizador alcanzó el máximo de 500 iteraciones; esto debe interpretarse como una observación técnica importante para la continuidad del proyecto, no como un error de ejecución.

9.1 Configuración mínima e implementación

```
mlp_model_1_hidden = MLPClassifier(hidden_layer_sizes=(100,), activation='relu', solver='adam', max_iter=500, random_state=42)
mlp_model_1_hidden.fit(X_train_scaled, y_train)
y_pred_mlp_1_hidden = mlp_model_1_hidden.predict(X_test_scaled)

mlp_model_2_hidden = MLPClassifier(hidden_layer_sizes=(100, 50), activation='relu', solver='adam', max_iter=500, random_state=42)
mlp_model_2_hidden.fit(X_train_scaled, y_train)
y_pred_mlp_2_hidden = mlp_model_2_hidden.predict(X_test_scaled)

mlp_model_3_hidden = MLPClassifier(hidden_layer_sizes=(100, 50, 25), activation='relu', solver='adam', max_iter=500, random_state=42)
mlp_model_3_hidden.fit(X_train_scaled, y_train)
y_pred_mlp_3_hidden = mlp_model_3_hidden.predict(X_test_scaled)
```

Arquitectura	hidden_layer_sizes	activation	solver	max_iter	Entrada
MLP 1 capa oculta	(100,)	relu	adam	500	X_train_scaled / X_test_scaled
MLP 2 capas ocultas	(100, 50)	relu	adam	500	X_train_scaled / X_test_scaled
MLP 3 capas ocultas	(100, 50, 25)	relu	adam	500	X_train_scaled / X_test_scaled

9.2 Resultados obtenidos e interpretación inicial

Modelo	Accuracy	Precision	Recall	F1-score	Matriz de confusión
MLP 1 capa oculta	0.8212	0.8182	0.7297	0.7714	[[93, 12], [20, 54]]
MLP 2 capas ocultas	0.8156	0.8154	0.7162	0.7626	[[93, 12], [21, 53]]
MLP 3 capas ocultas	0.8268	0.8116	0.7568	0.7832	[[92, 13], [18, 56]]

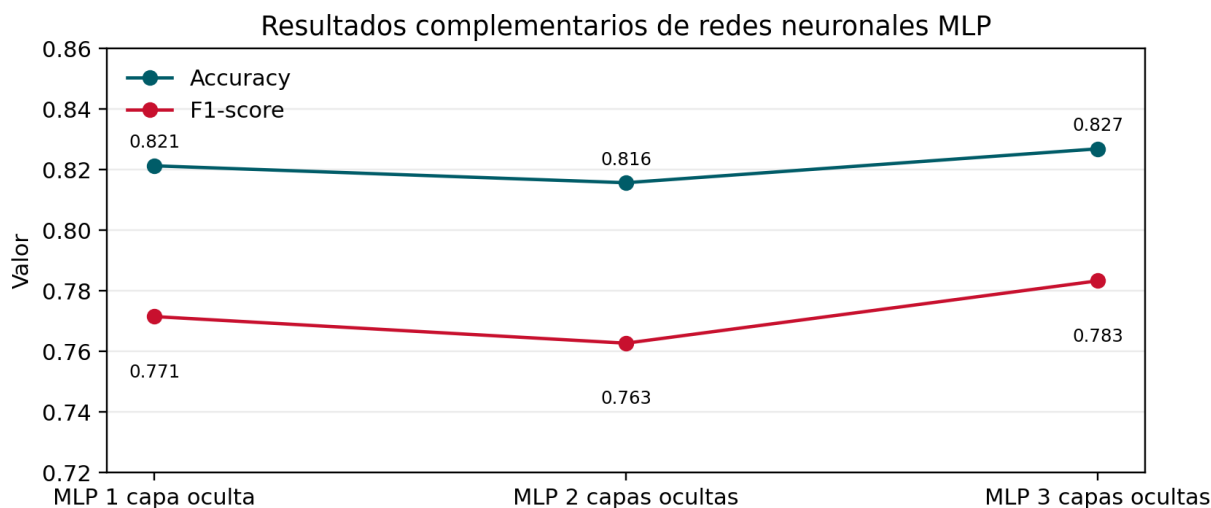


Figura 5. Comparación de accuracy y F1-score en arquitecturas MLP.

La red con una capa oculta obtiene $\text{accuracy}=0,8212$ y $\text{F1-score}=0,7714$, igualando el accuracy de SVM y superando su F1-score. La red con dos capas ocultas baja levemente a $\text{accuracy}=0,8156$ y $\text{F1-score}=0,7626$, lo que indica que agregar profundidad no mejora automáticamente el desempeño. La red con tres capas ocultas obtiene el mejor resultado global del cuaderno, con $\text{accuracy}=0,8268$ y $\text{F1-score}=0,7832$, además de 56 verdaderos positivos y 18 falsos negativos para la clase sobrevivió.

Desde una lectura técnica, el mejor desempeño de MLP con tres capas sugiere que la arquitectura más profunda captura patrones no lineales útiles. Sin embargo, la mejora respecto de SVM y MLP de una capa es acotada. Dado el tamaño moderado del dataset Titanic y la advertencia de convergencia, la recomendación es tratar este resultado como promisorio, pero no definitivo, y validarlo posteriormente con ajuste de hiperparámetros, regularización, early stopping o validación cruzada.

X. Evaluación preliminar de modelos de clasificación

La evaluación preliminar se basa en las matrices de confusión y en métricas derivadas de los resultados del notebook. Además de accuracy, se reportan precision, recall y F1-score para la clase positiva, porque el problema no debe interpretarse solo por aciertos globales: interesa distinguir falsos positivos y falsos negativos en la predicción de supervivencia.

Modelo	Accuracy	Precision	Recall	F1-score	Matriz de confusión
KNN	0.6983	0.6786	0.5135	0.5846	[[87, 18], [36, 38]]
Árbol de Decisión	0.7989	0.8276	0.6486	0.7273	[[95, 10], [26, 48]]
SVM	0.8212	0.8387	0.7027	0.7647	[[95, 10], [22, 52]]
Naive Bayes	0.7709	0.7200	0.7297	0.7248	[[84, 21], [20, 54]]
MLP 1 capa oculta	0.8212	0.8182	0.7297	0.7714	[[93, 12], [20, 54]]
MLP 2 capas ocultas	0.8156	0.8154	0.7162	0.7626	[[93, 12], [21, 53]]
MLP 3 capas ocultas	0.8268	0.8116	0.7568	0.7832	[[92, 13], [18, 56]]

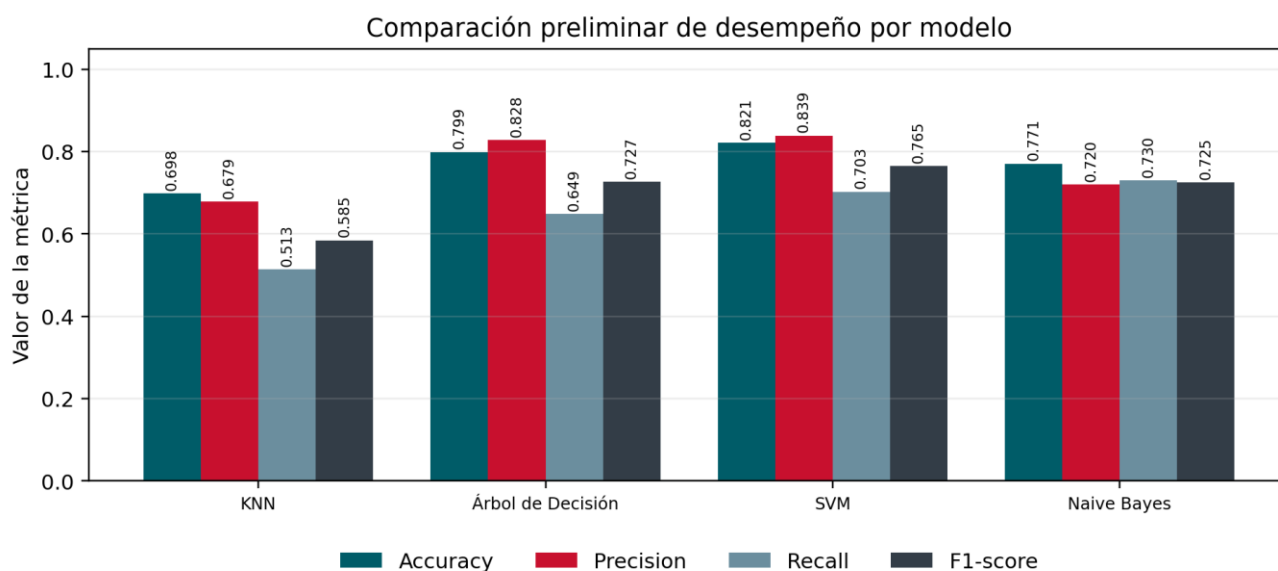


Figura 6. Comparación preliminar de métricas de clasificación en modelos clásicos.

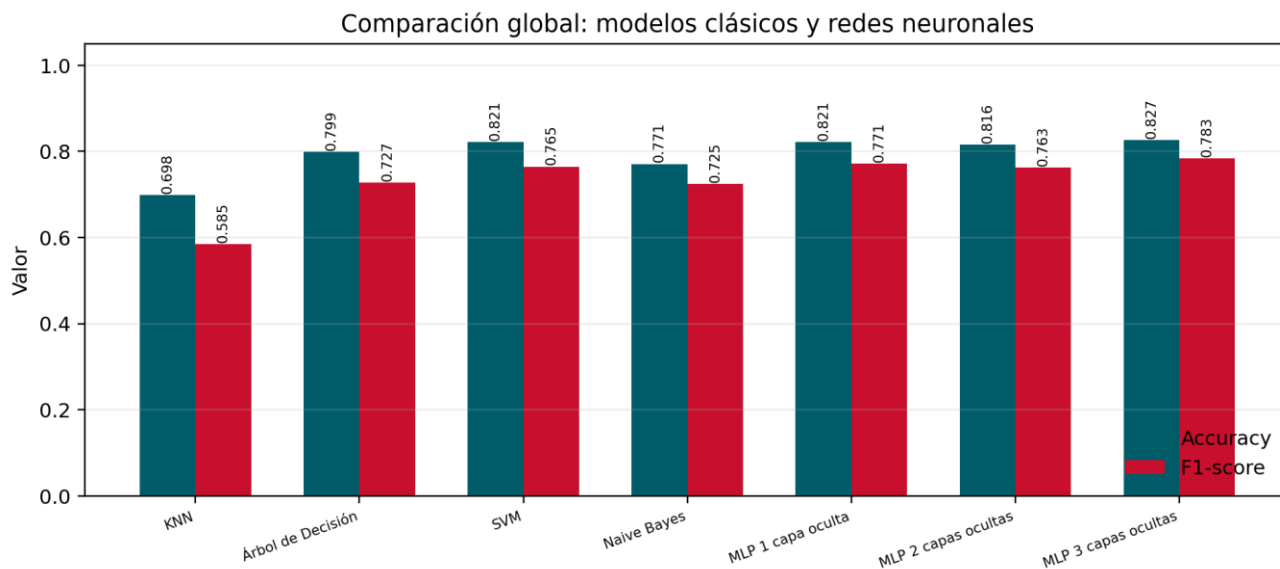


Figura 7. Comparación global de modelos clásicos y redes neuronales.

Matrices de confusión en conjunto de prueba

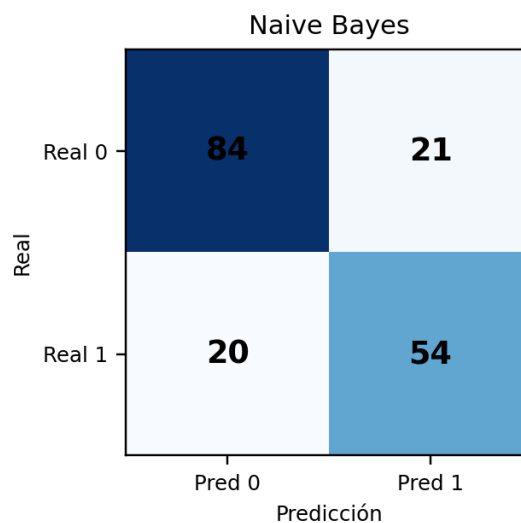
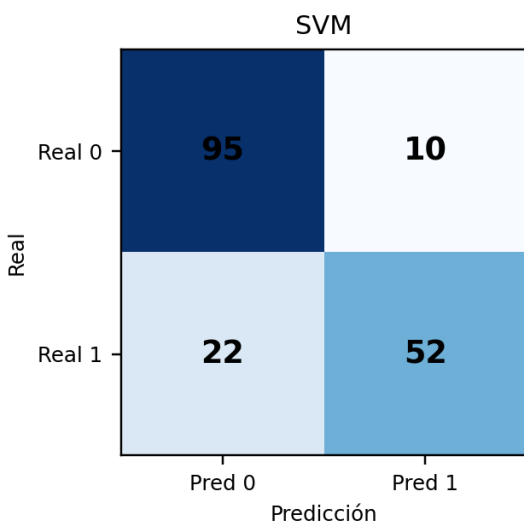
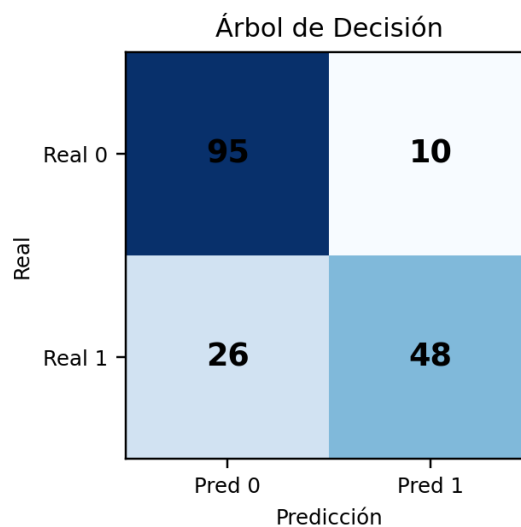
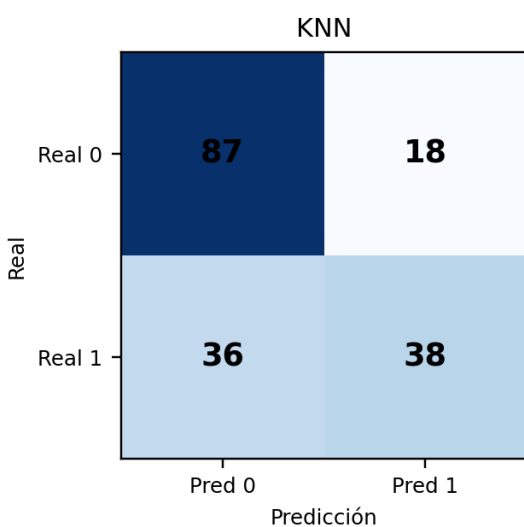


Figura 8. Matrices de confusión de los cuatro modelos exigidos por la actividad.

10.1 Comparación analítica inicial

Modelo	Fortalezas	Limitaciones	Lectura del resultado
KNN	Modelo simple y fácil de explicar por proximidad.	Menor accuracy y bajo recall; sensible a escalas y se entrenó sin escalamiento en el código.	Funciona como línea base, pero no es el modelo más recomendable en esta ejecución.
Árbol de Decisión	Interpretable mediante reglas; buena precisión.	Recall moderado; puede depender de profundidad y criterios de división.	Mejora a KNN y ofrece explicación, aunque no alcanza a SVM ni a MLP.
SVM	Mejor desempeño entre los cuatro modelos clásicos exigidos; buena precisión y recall.	Menor interpretabilidad; requiere escalamiento y ajuste de hiperparámetros.	Modelo clásico preliminar recomendado por desempeño global.
Naïve Bayes	Eficiente y con recall competitivo.	Supuesto de independencia condicional puede ser poco realista.	Útil como contraste probabilístico, no como mejor modelo final.
MLP 1 capa oculta	Iguala el accuracy de SVM y mejora F1-score.	Presenta advertencia de convergencia; menor interpretabilidad.	Alternativa avanzada competitiva con arquitectura simple.
MLP 2 capas ocultas	Aumenta capacidad representacional.	No mejora frente a una capa; mayor complejidad no garantiza mejor resultado.	Debe revisarse con tuning antes de preferirla.
MLP 3 capas ocultas	Mejor accuracy y F1 global del cuaderno.	Más compleja; riesgo de sobreajuste y convergencia no completamente cerrada.	Mejor modelo global preliminar, sujeto a validación posterior.

La lectura comparativa tiene dos niveles. Entre los cuatro modelos exigidos explícitamente por la actividad, SVM es la alternativa más sólida. Al incorporar las redes neuronales del cuaderno complementario, MLP con tres capas ocultas obtiene el mejor desempeño global, con accuracy=0,8268 y F1-score=0,7832. Esta mejora debe interpretarse con cautela, porque el aumento de complejidad también incrementa la necesidad de control de sobreajuste, convergencia y validación cruzada.

XI. Conclusiones

La Fase 3 permitió construir una implementación inicial de modelos de clasificación supervisada sobre el dataset Titanic. El proceso incluyó carga de datos, limpieza, imputación, codificación de variables categóricas, separación entre predictores y variable objetivo, partición entrenamiento/prueba y evaluación mediante matrices de confusión. Esta secuencia cumple el propósito de fortalecer el pipeline analítico antes de la entrega final de la fase.

Los resultados muestran diferencias claras entre clasificadores. KNN obtiene accuracy=0,6983 y F1=0,5846, por lo que su desempeño es el más débil en esta ejecución. El árbol de decisión alcanza accuracy=0,7989 y F1=0,7273, mejorando el rendimiento y aportando interpretabilidad. Naïve Bayes obtiene accuracy=0,7709 y F1=0,7248, con un recall competitivo para la clase sobrevivió. SVM logra el mejor desempeño entre los cuatro modelos clásicos exigidos, con accuracy=0,8212 y F1=0,7647. Al incorporar redes neuronales, MLP con tres capas ocultas obtiene el mejor resultado global, con accuracy=0,8268 y F1=0,7832.

La conclusión preliminar es que el proyecto debe conservar dos lecturas complementarias: SVM como mejor modelo clásico exigido por la actividad y MLP de tres capas como mejor alternativa global del cuaderno ampliado. Sin embargo, el árbol de decisión debe mantenerse como alternativa interpretable, Naïve Bayes como contraste probabilístico eficiente y KNN como línea base que requiere revisión metodológica, particularmente por su sensibilidad al escalamiento.

Para el informe final de la Fase 3 se recomienda ampliar la evaluación con validación cruzada, búsqueda de hiperparámetros, comparación de umbrales de decisión y análisis de casos mal clasificados. También conviene revisar el pipeline para evitar inconsistencias de escalamiento entre modelos, sin modificar retroactivamente la evidencia entregada para esta actividad formativa.

Criterio de cierre	Resultado de la fase
Mejor modelo clásico requerido	SVM.
Mejor modelo global con redes	MLP con tres capas ocultas.
Métrica decisiva	Mayor accuracy y mayor F1-score global en MLP 3 capas.
Principal fortaleza	Capacidad para capturar relaciones no lineales entre variables y supervivencia.
Principal limitación	Menor interpretabilidad y advertencia de convergencia en redes neuronales.
Proyección	Validación cruzada, tuning, early stopping, regularización y análisis de errores por clase.

XII. Uso de fuentes, citación y referencias

El informe utiliza fuentes docentes de la asignatura, documentación técnica oficial y referencias académicas complementarias. Las fuentes se integran en el desarrollo para sustentar la descripción conceptual de los modelos, la interpretación del pipeline y el uso de clases de scikit-learn.

Alpaydin, E. (2010). Introduction to machine learning. MIT Press.

Géron, A. (2019). Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow. O'Reilly Media.

James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). An introduction to statistical learning: With applications in R (2nd ed.). Springer.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

Ruete, D. (2026a). k-Nearest Neighbors (k-NN) [Apunte]. Universidad Andrés Bello, Santiago, Chile.

Ruete, D. (2026b). Árboles de decisión [Apunte]. Universidad Andrés Bello, Santiago, Chile.

Ruete, D. (2026c). Máquinas de Vectores de Soporte (SVM) [Apunte]. Universidad Andrés Bello, Santiago, Chile.

Ruete, D. (2026d). Naive Bayes [Apunte]. Universidad Andrés Bello, Santiago, Chile.

Ruete, D. (2026e). Redes Neuronales [Apunte]. Universidad Andrés Bello, Santiago, Chile.

Ruete, D. (2026f). Semana 03. Formativa 3: Avance proyecto Fase 3 [Instrucciones de actividad]. Universidad Andrés Bello, Santiago, Chile.

Ruete, D. (2026g). Semana 3: Implementación de modelos de clasificación supervisada [Recurso de plataforma]. Universidad Andrés Bello, Santiago, Chile.

scikit-learn Developers. (2026a). sklearn.neighbors.KNeighborsClassifier. scikit-learn documentation. <https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html>

scikit-learn Developers. (2026b). sklearn.tree.DecisionTreeClassifier. scikit-learn documentation. <https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html>

scikit-learn Developers. (2026c). sklearn.svm.SVC. scikit-learn documentation. <https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html>

scikit-learn Developers. (2026d). sklearn.naive_bayes.GaussianNB. scikit-learn documentation. https://scikit-learn.org/stable/modules/generated/sklearn.naive_bayes.GaussianNB.html

scikit-learn Developers. (2026e). sklearn.neural_network.MLPClassifier. scikit-learn documentation. https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html

XIII. Anexos

Anexo A. Evidencia resumida del código fuente

Los siguientes fragmentos sintetizan secciones del notebook que producen evidencias principales del informe. El equipo debe conservar el notebook completo en el repositorio y verificar que la última ejecución coincida con los valores reportados.

```
titanic_df = sns.load_dataset('titanic')
titanic_df_processed = titanic_df.copy()
columns_to_drop = ['deck', 'class', 'who', 'adult_male', 'embark_town', 'alive', 'alone']
titanic_df_processed = titanic_df_processed.drop(columns=columns_to_drop)
titanic_df_processed['age'].fillna(titanic_df_processed['age'].median(), inplace=True)
titanic_df_processed['embarked'].fillna(titanic_df_processed['embarked'].mode()[0], inplace=True)
titanic_df_processed = pd.get_dummies(titanic_df_processed, columns=['sex', 'embarked'], drop_first=True)
```

Anexo B. Resultados complementarios de redes neuronales

El notebook también implementa redes neuronales MLP con una, dos y tres capas ocultas. Estos resultados se presentan como evidencia complementaria, porque la instrucción específica de la actividad solicita KNN, árbol de decisión, SVM y Naïve Bayes como modelos principales.

Modelo complementario	Accuracy	Precision	Recall	F1-score	Matriz de confusión
MLP 1 capa oculta	0.8212	0.8182	0.7297	0.7714	[[93, 12], [20, 54]]
MLP 2 capas ocultas	0.8156	0.8154	0.7162	0.7626	[[93, 12], [21, 53]]
MLP 3 capas ocultas	0.8268	0.8116	0.7568	0.7832	[[92, 13], [18, 56]]

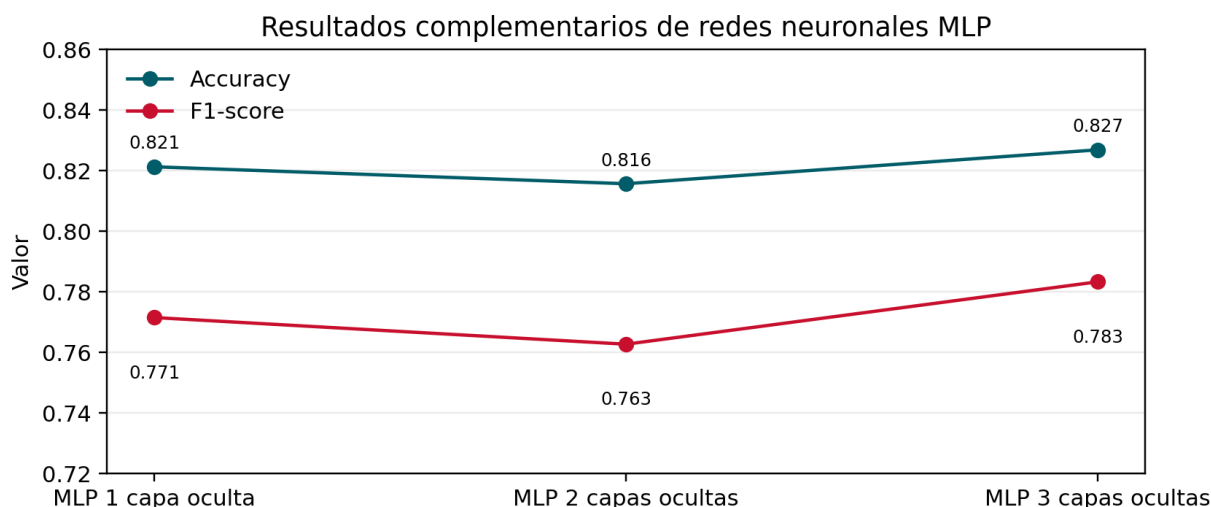


Figura 9. Resultados complementarios de redes neuronales MLP.